

PSTAT_135_Final_Project (Python)

[Import notebook](#)

PSTAT 135 Final Project

Chrtistina Nassar, Hongjun Kim, Mathhew Champagne, Yuhan Lin

This notebook explores patterns of voter participation in Wyoming using data from the U.S. national voter file. We apply both supervised and unsupervised learning techniques to investigate how demographic factors, particularly race, age, and homeownership, relate to general election turnout.

In the supervised analysis, we predict whether an individual voted in the 2018 general election based on their race and age group, examining how these features influence turnout independently and interactively.

In the unsupervised analysis, we use clustering to identify distinct demographic groupings across race, age, and homeownership status. We then analyze voting behavior within each cluster to reveal underlying patterns that may not be captured by single variables.

Our goal is to uncover actionable insights that could inform data-driven strategies for civic engagement and voter outreach.

1. Data in BigQuery

1.2 Shell Script in Cloud Shell

We ran the shell script below and returned the count for number of rows in each dataset.

```
!/bin/bash
```

```
''' Define array of state codes ''' states=(AK AL AR AZ CO CT DC DE GA HI IA ID IL IN KS KY LA MA MD ME MI MN MO
MS MT NC ND NE NH NJ NM NV OH OK OR RI SC SD TN UT VA VT WA WI WV WY)
```

```
''' Loop through each state and run the BigQuery row count query '''
```

```
for state in "${states[@]}; do
```

```
echo "Running query for $state..."
```

```
bq query --use_legacy_sql=false \
```

```
"SELECT COUNT(*) AS count FROM `pstat-135-235-spring-2025.voterfile.$state`;"
```

```
done
```

```

+-----+ | count | +-----+ | 548259 | +-----+ +-----+ | count | +-----+ | 3458534 | +-----+ +-----+
+ | count | +-----+ | 1647730 | +-----+ +-----+ | count | +-----+ | 4266400 | +-----+ +-----+ |
count | +-----+ | 3744871 | +-----+ +-----+ | count | +-----+ | 2373153 | +-----+ +-----+ | count |
+-----+ | 473913 | +-----+ +-----+ | count | +-----+ | 707643 | +-----+ +-----+ | count | +-----+ |
7105478 | +-----+ +-----+ | count | +-----+ | 792412 | +-----+ +-----+ | count | +-----+ | 2100358 |
+-----+ +-----+ | count | +-----+ | 946908 | +-----+ +-----+ | count | +-----+ | 8336875 | +-----+
+ +-----+ | count | +-----+ | 4396576 | +-----+ +-----+ | count | +-----+ | 1795288 | +-----+ +--
-----+ | count | +-----+ | 3207343 | +-----+ +-----+ | count | +-----+ | 2984341 | +-----+ +-----+
-+ | count | +-----+ | 4572639 | +-----+ +-----+ | count | +-----+ | 4110570 | +-----+ +-----+ |
count | +-----+ | 1040452 | +-----+ +-----+ | count | +-----+ | 7593651 | +-----+ ^C+-----+ |
count | +-----+ | 4038043 | +-----+ +-----+ | count | +-----+ | 1983032 | +-----+

```

This shell script automates the process of counting the number of rows in each U.S. state's voter file stored in BigQuery. It defines an array of state codes and loops through them one by one. For each state, it constructs and executes a SQL query using the bq command-line tool to count the number of records in the corresponding BigQuery table. The script prints a message indicating which state's data is currently being queried, and then outputs the row count result.

1.3. Voter Turnout Query

1.4. BigQuery Notebook

The notebook runs on BigQuery's managed serverless infrastructure, so compute is auto-scaled and does not require provisioning. Based on the job logs, the region used was likely us, but this can be confirmed via the "Open Job" page. From that page, we also learned that one query processed ~980 MB of data, while another used cached results (0 bytes processed). The job page provides insights into performance, resource usage, and potential bottlenecks.

Approaches 1.3 and 1.4 offer two different ways to analyze voter turnout using BigQuery, each with its own strengths. In 1.3, SQL queries are written directly in BigQuery Studio or within a notebook using %%bigquery, allowing for fast and efficient aggregation of large datasets without moving data outside of BigQuery. This method is ideal for tasks like calculating turnout proportions by year, since the operations are simple and optimized for SQL. It also conserves memory and offers features like result caching and easy job tracking. In contrast, approach 1.4 uses the bigframes.pandas Python API to load full tables into a pandas-like DataFrame, which allows for more flexible and familiar Python-based analysis. This is especially useful when creating visualizations or applying machine learning models. However, it can be slower or memory-intensive if the dataset is large, and is less efficient for simple aggregations. Overall, 1.3 is better for efficient, large-scale querying, while 1.4 is better suited for exploratory data analysis and custom logic in Python.

2. Duplicate Data into Databricks

2.1. Choose a state and export the data to your Databricks storage location (Wyoming):

Create a managed volume in Databricks & Note the Storage Location under Details

Extract table to storage location. Refer to documentation on GCP documentation. Use Parquet format, set SNAPPY compression, and print header to true

2.2. Create a Databricks table from exported files in storage location.

```
df = spark.read.format("csv").load("gs://final_project_bucket_135_2/wy.csv")

df.show()
df.printSchema()
```

df: pyspark.sql.dataframe.DataFrame = [_c0: string]

```
|189658\tLALWY1778...|
|189659\tLALWY4105...|
|189660\tLALWY6091...|
|189661\tLALWY6091...|
|189662\tLALWY4105...|
|189663\tLALWY4105...|
|189664\tLALWY1778...|
|189665\tLALWY5046...|
|189666\tLALWY5046...|
|189667\tLALWY6092...|
|189668\tLALWY1779...|
|189669\tLALWY1778...|
|189670\tLALWY5046...|
|189671\tLALWY5046...|
|189672\tLALWY6049...|
|189673\tLALWY6049...|
```

+-----+

only showing top 20 rows

root

```
|-- _c0: string (nullable = true)
```

```
df.write.format("csv") \
    .mode("overwrite") \
    .save("/Volumes/assignment2/default/voterfile-export/WY")
```

2.3. EDA and Visualization with Databricks

1. Use at least 5 different Pyspark Pandas APIs to perform various data exploration tasks. Explain the documentation specification

```
import pyspark.pandas as ps

# read from table
psdf = ps.read_table("assignment2.default.WY")
psdf.head(5)
```

```
import pyspark.pandas as ps
df = spark.table("WY") # Load Spark DataFrame
df_API = df.pandas_api() # Convert to pandas-on-Spark DataFrame
```

df: pyspark.sql.dataframe.DataFrame = [SEQUENCE: long, LALVOTERID: string ... 439 more fields]

```
# API #1
df_API.info()
# Basic structure and types: Shows total Non-Null Count of each column and Data Type: object, int32, bool,
int64
```

421	PresidentialPrimary_2004	0 non-null	object
422	AnyElection_2003	0 non-null	object
423	General_2002	0 non-null	object
424	Primary_2002	0 non-null	object
425	AnyElection_2001	0 non-null	object
426	General_2000	0 non-null	object
427	Primary_2000	0 non-null	object
428	PresidentialPrimary_2000	0 non-null	object
429	PRI_BLT_2020	135406 non-null	object
430	PRI_BLT_2018	123280 non-null	object
431	PRI_BLT_2016	96355 non-null	object
432	PRI_BLT_2014	92634 non-null	object
433	PRI_BLT_2012	77979 non-null	object
434	PRI_BLT_2010	90815 non-null	object
435	PRI_BLT_2008	66334 non-null	object
436	PRI_BLT_2006	0 non-null	object
437	PRI_BLT_2004	0 non-null	object
438	PRI_BLT_2002	0 non-null	object
439	PRI_BLT_2000	0 non-null	object
440	_rescued_data	0 non-null	object

dtypes: bool(1), float64(2), int64(70), object(368)

```
# API #2
df_API.describe()
# Summary Statistics
```

```
# API #3
display(df_API.groupby("Voters_Gender").size().reset_index(name="count"))
# Counts observations sum for each gender
```

Table

```
# API #4
df_API.corr()
# Correlation between columns Creates matrix that compares correlation between each variables
# Works on only numerical values non-numeric are ignored
# Values range from 1 to -1 depending on correlation
#
#          SEQUENCE      VoterTelephones_LandlineAreaCode
#SEQUENCE      1.000000      -0.031363
#VoterTelephones_LandlineAreaCode      -0.031363      1.000000
```

```
# API #5
df_API.isnull().sum()
# Missing values analysis Returns total sum of missing values per column
```

```
PresidentialPrimary_2004      290408
AnyElection_2003              290408
General_2002                  290408
Primary_2002                  290408
AnyElection_2001              290408
General_2000                  290408
Primary_2000                  290408
PresidentialPrimary_2000      290408
PRI_BLT_2020                  155002
PRI_BLT_2018                  167128
PRI_BLT_2016                  194053
PRI_BLT_2014                  197774
PRI_BLT_2012                  212429
PRI_BLT_2010                  199593
PRI_BLT_2008                  224074
PRI_BLT_2006                  290408
PRI_BLT_2004                  290408
PRI_BLT_2002                  290408
PRI_BLT_2000                  290408
_rescued_data                  290408
dtype: int64
```

2. Visualize voter turnout by age groups: 20s, 30s, ..., 80+. Each age groups is to be plotted as a line trace in a shared plot

```
import pandas as pd
import matplotlib.pyplot as plt
from pyspark.sql import functions as F
bins = [20, 30, 40, 50, 60, 70, 80, 150]
labels = ['20s', '30s', '40s', '50s', '60s', '70s', '80+']
```

```
# Age Classification
df = df.withColumn('Age_Group', F.when((df['Voters_Age'] >= 20) & (df['Voters_Age'] < 30), '20s')
    .when((df['Voters_Age'] >= 30) & (df['Voters_Age'] < 40), '30s')
    .when((df['Voters_Age'] >= 40) & (df['Voters_Age'] < 50), '40s')
    .when((df['Voters_Age'] >= 50) & (df['Voters_Age'] < 60), '50s')
    .when((df['Voters_Age'] >= 60) & (df['Voters_Age'] < 70), '60s')
    .when((df['Voters_Age'] >= 70) & (df['Voters_Age'] < 80), '70s')
    .otherwise('80+') )
```

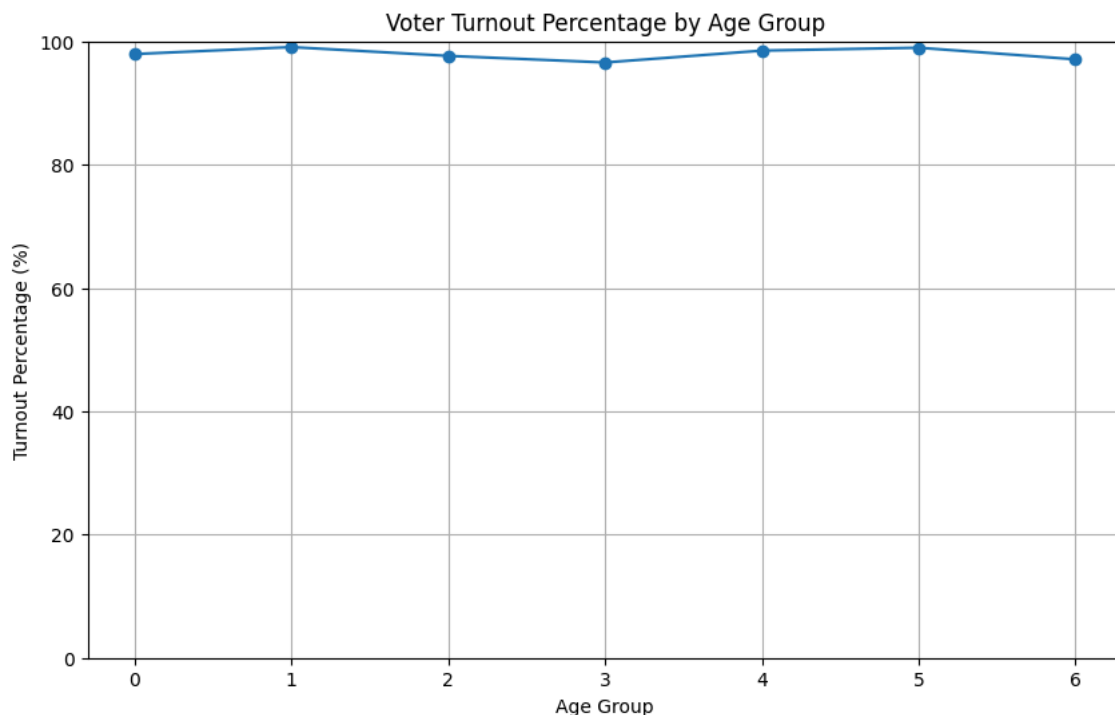
df: pyspark.sql.dataframe.DataFrame = [SEQUENCE: long, LALVOTERID: string ... 440 more fields]

```
# Count active and inactive per age group
turnout = df.groupBy(['Age_Group',
    'Voters_Active']).count().groupBy('Age_Group').pivot('Voters_Active').agg(F.first('count')).fillna(0)

# Calculate turnout percentage = Active / Total * 100
turnout = turnout.withColumn('Turnout_Percentage', F.when((F.col('A').isNotNull() & F.col('I').isNotNull()),
    (F.col('A') / (F.col('A') + F.col('I'))) * 100).otherwise(0))
```

turnout: pyspark.sql.dataframe.DataFrame = [Age_Group: string, A: long ... 2 more fields]

```
# Plot
plt.figure(figsize=(10,6))
plt.plot(turnout.toPandas().index, turnout.toPandas()['Turnout_Percentage'], marker='o')
plt.title('Voter Turnout Percentage by Age Group')
plt.xlabel('Age Group')
plt.ylabel('Turnout Percentage (%)')
plt.ylim(0, 100)
plt.grid(True)
plt.show()
```



3. Create a choropleth visualization of voter turnout rate by county for a chosen year.

```
from pyspark.sql.functions import col, when, sum, count, row_number, lit

# Load voterfile table
df_2018 = df.withColumn("voted_2018", when(col("General_2018") == "Y", 1).otherwise(0))
turnout_df = df_2018.groupBy("Voters_FIPS") \
    .agg(
        (sum("voted_2018") / count("*")).alias("turnout_rate")
    )

turnout_pd = turnout_df.toPandas()
turnout_pd['fips'] = (turnout_pd['Voters_FIPS'].astype(int) + 56000).astype(str)
```

- ▶  df_2018: pyspark.sql.dataframe.DataFrame = [SEQUENCE: long, LALVOTERID: string ... 441 more fields]
- ▶  turnout_df: pyspark.sql.dataframe.DataFrame = [Voters_FIPS: long, turnout_rate: double]
- ▶  turnout_pd: pandas.core.frame.DataFrame = [Voters_FIPS: int64, turnout_rate: float64 ... 1 more field]

```
import plotly.express as px

fig = px.choropleth(
    turnout_pd,
    geojson="https://raw.githubusercontent.com/plotly/datasets/master/geojson-counties-fips.json",
    locations="fips",
    color="turnout_rate",
    color_continuous_scale="Blues",
    scope="usa",
    labels={'turnout_rate': 'Turnout Rate'})

fig.update_geos(fitbounds="locations", visible=False)
fig.update_layout(title="Voter Turnout - Wyoming (2018)", margin={"r":0,"t":30,"l":0,"b":0})
```

4. Create a choropleth visualization of party affiliation by county. (/url)

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

# Count number of voters per party per county
party_counts = df.groupBy("Voters_FIPS", "Parties_Description").count()

# Get the top party in each county
window = Window.partitionBy("Voters_FIPS").orderBy(F.desc("count"))

top_party = party_counts.withColumn("rank", F.row_number().over(window)) \
    .filter(F.col("rank") == 1) \
    .select("Voters_FIPS", "Parties_Description")
```

- ▶  party_counts: pyspark.sql.dataframe.DataFrame = [Voters_FIPS: long, Parties_Description: string ... 1 more field]

top_party: pyspark.sql.dataframe.DataFrame = [Voters_FIPS: long, Parties_Description: string]

```
party_pd = top_party.toPandas()
party_pd['fips'] = "56" + party_pd['Voters_FIPS'].astype(str).str.zfill(3)
```

party_pd: pandas.core.frame.DataFrame = [Voters_FIPS: int64, Parties_Description: object ... 1 more field]

```
import plotly.express as px

fig = px.choropleth(
    party_pd,
    geojson="https://raw.githubusercontent.com/plotly/datasets/master/geojson-counties-fips.json",
    locations="fips",
    color="Parties_Description",
    scope="usa",
    color_discrete_sequence=px.colors.qualitative.Set3,
    labels={"Parties_Description": "Dominant Party"},
    title="Dominant Party Affiliation by County (Wyoming)"
)

fig.update_geos(fitbounds="locations", visible=False)
fig.update_layout(margin={"r":0,"t":30,"l":0,"b":0})
fig.show()
```

3. Supervised and Unsupervised Learning

1. (Supervised) How does general election turnout vary across racial groups and age brackets within Wyoming, and what patterns emerge when these factors intersect?
2. (Unsupervised) What distinct groups or patterns emerge when clustering Wyoming residents based on homeownership status, racial background, and other demographic features — and how do these clusters differ in their voting behavior?


```

from pyspark.sql.functions import col, when, avg
import seaborn as sns
import matplotlib.pyplot as plt

# Define a consistent color palette
race_palette = {
    "White": "#1f77b4",
    "Black": "#ff7f0e",
    "Hispanic": "#2ca02c",
    "Asian": "#d62728",
    "Other": "#9467bd"
}

# Binary column for active voters
df = df.withColumn("is_active", when(col("Voters_Active") == "A", 1).otherwise(0))

# Age group buckets
df = df.withColumn("age_group", when(col("Voters_Age") < 30, "18-29")
                                   .when(col("Voters_Age") < 45, "30-44")
                                   .when(col("Voters_Age") < 60, "45-59")
                                   .otherwise("60+"))

# Simplify race/ethnic groups
df = df.withColumn("race_group",
    when(col("EthnicGroups_EthnicGroup1Desc").rlike("(?i)Hispanic"), "Hispanic")
    .when(col("EthnicGroups_EthnicGroup1Desc").rlike("(?i)African|Black"), "Black")
    .when(col("EthnicGroups_EthnicGroup1Desc").rlike("(?i)Asian"), "Asian")
    .when(col("EthnicGroups_EthnicGroup1Desc").rlike("(?i)European|White"), "White")
    .otherwise("Other")
)

# Group and compute average turnout rate (proportion active)
turnout_df = df.groupBy("age_group", "race_group") \
    .agg(avg("is_active").alias("turnout_rate"))

# Convert to pandas for plotting
turnout_pd = turnout_df.toPandas()

```

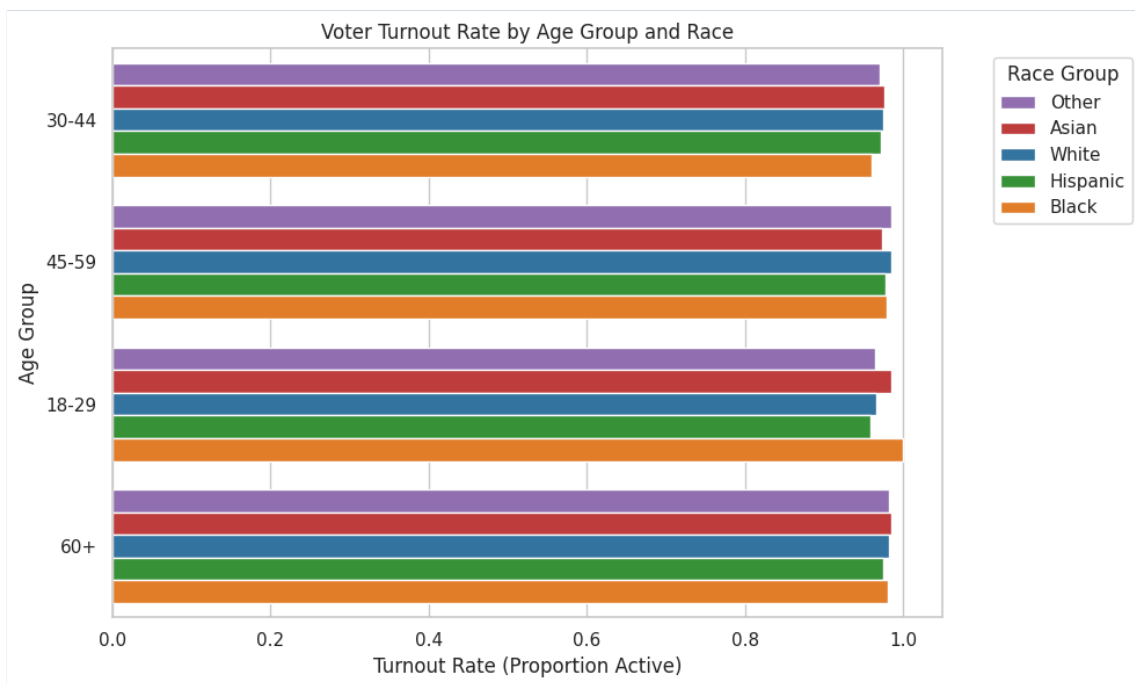
- ▶  df: pyspark.sql.dataframe.DataFrame = [SEQUENCE: long, LALVOTERID: string ... 442 more fields]
- ▶  turnout_df: pyspark.sql.dataframe.DataFrame = [age_group: string, race_group: string ... 1 more field]
- ▶  turnout_pd: pandas.core.frame.DataFrame = [age_group: object, race_group: object ... 1 more field]

```

# Plot
sns.set(style="whitegrid")
plt.figure(figsize=(10, 6))
sns.barplot(
    data=turnout_pd,
    y="age_group",
    x="turnout_rate",
    hue="race_group",
    palette=race_palette
)

plt.xlabel("Turnout Rate (Proportion Active)")
plt.ylabel("Age Group")
plt.title("Voter Turnout Rate by Age Group and Race")
plt.legend(title="Race Group", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()

```



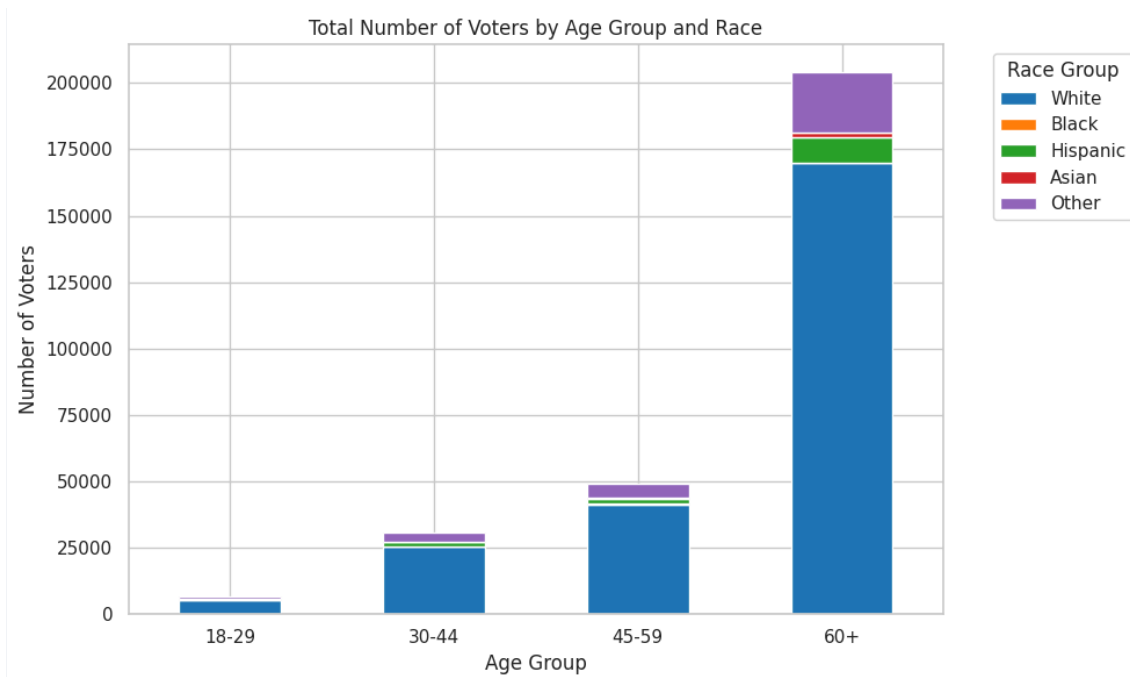
```
voter_counts_df = df.groupBy("age_group", "race_group").count()
voter_counts_pd = voter_counts_df.toPandas()

# Sort age_group in order
age_order = ["18-29", "30-44", "45-59", "60+"]
voter_counts_pd["age_group"] = pd.Categorical(voter_counts_pd["age_group"], categories=age_order,
ordered=True)

voter_pivot = voter_counts_pd.pivot(index="age_group", columns="race_group", values="count").fillna(0)
voter_pivot = voter_pivot[race_palette.keys()] # Ensure correct column order

# Plot
voter_pivot.plot(kind="bar", stacked=True, figsize=(10, 6), color=[race_palette[r] for r in
voter_pivot.columns])
plt.xlabel("Age Group")
plt.ylabel("Number of Voters")
plt.title("Total Number of Voters by Age Group and Race")
plt.legend(title="Race Group", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

- ▶ voter_counts_df: pyspark.sql.dataframe.DataFrame = [age_group: string, race_group: string ... 1 more field]
- ▶ voter_counts_pd: pandas.core.frame.DataFrame = [age_group: category, race_group: object ... 1 more field]
- ▶ voter_pivot: pandas.core.frame.DataFrame = [White: int64, Black: int64 ... 3 more fields]

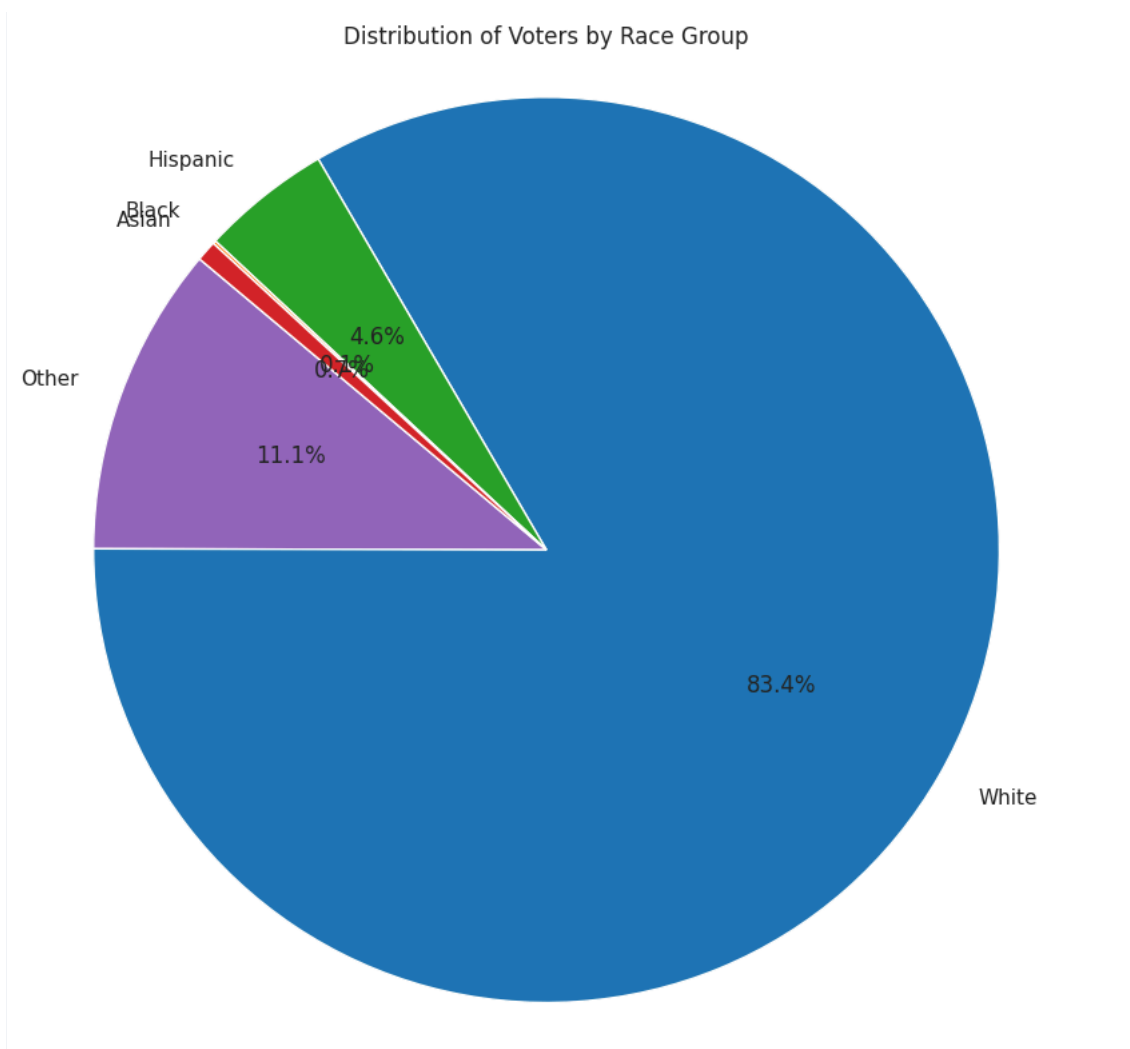


```
# Group by race_group to get total voters per race
race_counts_df = df.groupBy("race_group").count()
race_counts_pd = race_counts_df.toPandas()
race_counts_pd = race_counts_pd[race_counts_pd["race_group"].isin(race_palette.keys())]
colors = [race_palette[r] for r in race_counts_pd["race_group"]]

#Plot
plt.figure(figsize=(8, 8))
plt.pie(
    race_counts_pd["count"],
    labels=race_counts_pd["race_group"],
    autopct="%1.1f%%",
    startangle=140,
    colors=colors
)

plt.title("Distribution of Voters by Race Group")
plt.axis("equal")
plt.tight_layout()
plt.show()
```

- ▶ race_counts_df: pyspark.sql.dataframe.DataFrame = [race_group: string, count: long]
- ▶ race_counts_pd: pandas.core.frame.DataFrame = [race_group: object, count: int64]



```
import seaborn as sns
import matplotlib.pyplot as plt

# Filter DataFrame to include only active voters
active_df = df.filter(col("Voters_Active") == "A")
active_pd = active_df.select("Voters_Age", "race_group").toPandas()

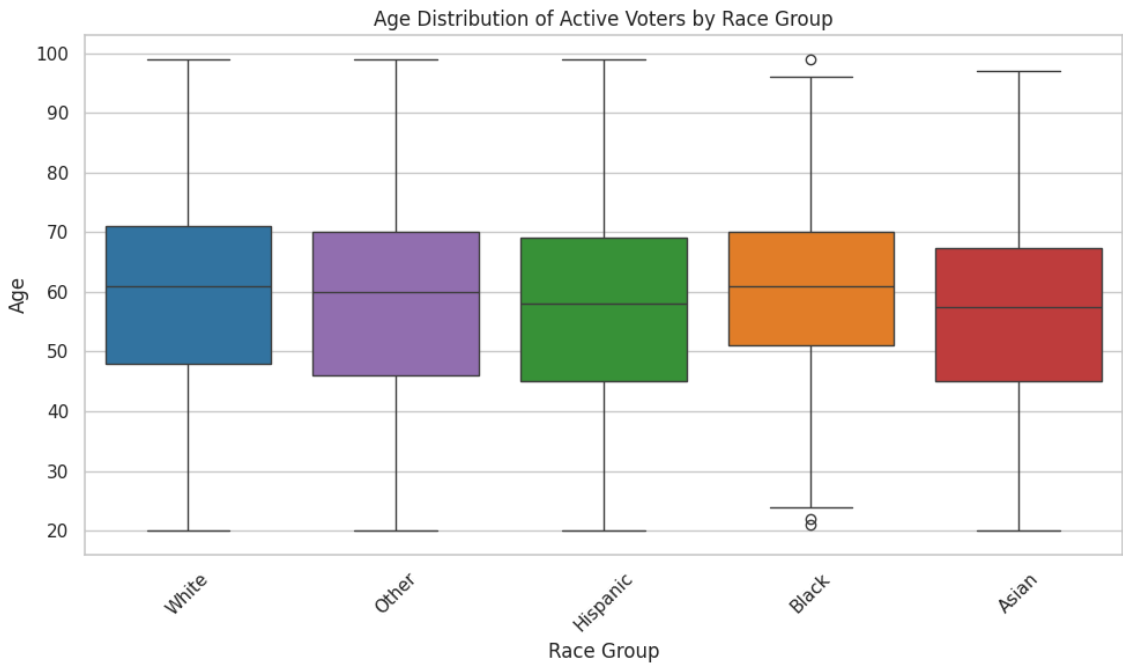
plt.figure(figsize=(10, 6))
sns.boxplot(
    data=active_pd,
    x="race_group",
    y="Voters_Age",
    palette=race_palette
)

plt.xlabel("Race Group")
plt.ylabel("Age")
plt.title("Age Distribution of Active Voters by Race Group")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

- ▶ active_df: pyspark.sql.dataframe.DataFrame = [SEQUENCE: long, LALVOTERID: string ... 442 more fields]
- ▶ active_pd: pandas.core.frame.DataFrame = [Voters_Age: float64, race_group: object]

```
/root/.ipykernel/14287/command-1289750558566045-740024902:9: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.



```

# Select general election columns and create year-turnout pairs
from pyspark.sql.functions import expr, when

years = [2004, 2006, 2008, 2010, 2012, 2014, 2016, 2018]
for year in years:
    df = df.withColumn(f"voted_{year}", when(col(f"General_{year}") == 'Y', 1).otherwise(0))

#Aggregate turnout by race and year
from pyspark.sql import DataFrame

def aggregate_turnout(df: DataFrame, group_col: str) -> DataFrame:
    aggs = []
    for year in years:
        aggs.append(
            (sum(f"voted_{year}") / count("*")).alias(f"turnout_{year}")
        )
    return df.groupBy(group_col).agg(*aggs)

race_trend_df = aggregate_turnout(df, "race_group")

# Convert to Pandas for plotting
race_trend_pd = race_trend_df.toPandas()

# Reshape data for seaborn line plot
race_trend_melted = race_trend_pd.melt(id_vars="race_group",
                                       value_vars=[f"turnout_{y}" for y in years],
                                       var_name="year", value_name="turnout_rate")

# Clean year strings
race_trend_melted["year"] = race_trend_melted["year"].str.extract("(\\d{4})").astype(int)

```

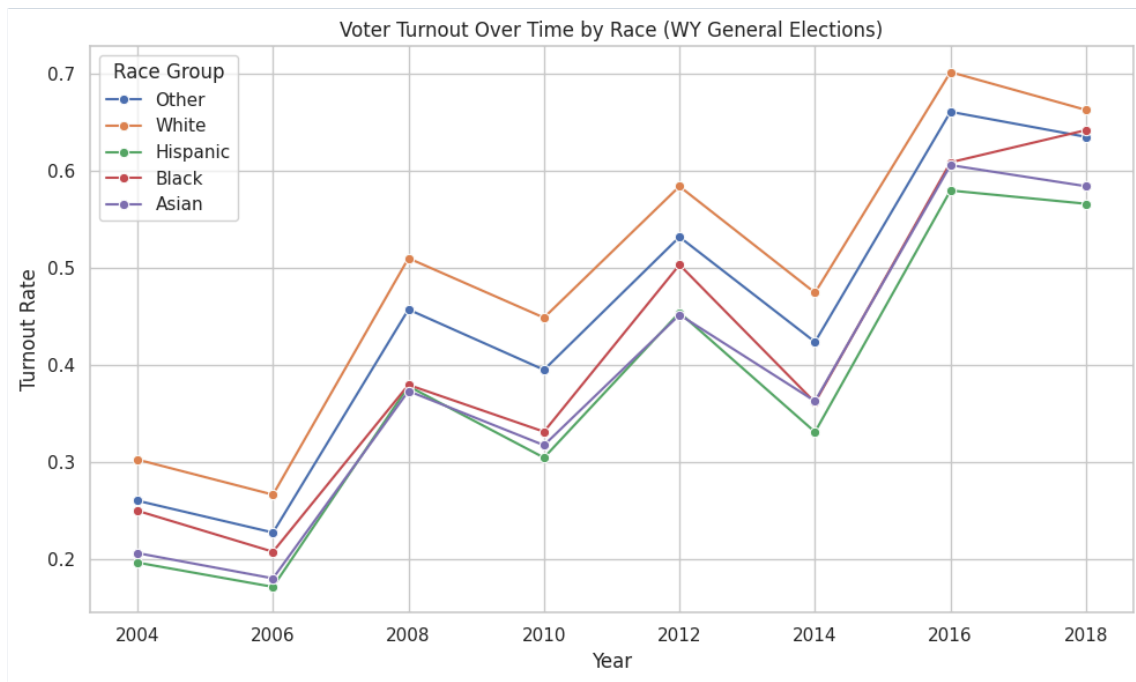
- ▶  df: pyspark.sql.dataframe.DataFrame = [SEQUENCE: long, LALVOTERID: string ... 450 more fields]
- ▶  race_trend_df: pyspark.sql.dataframe.DataFrame = [race_group: string, turnout_2004: double ... 7 more fields]
- ▶  race_trend_melted: pandas.core.frame.DataFrame = [race_group: object, year: int64 ... 1 more field]
- ▶  race_trend_pd: pandas.core.frame.DataFrame = [race_group: object, turnout_2004: float64 ... 7 more fields]

```

#Plot
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
sns.lineplot(data=race_trend_melted, x="year", y="turnout_rate", hue="race_group", marker="o")
plt.title("Voter Turnout Over Time by Race (WY General Elections)")
plt.ylabel("Turnout Rate")
plt.xlabel("Year")
plt.xticks(years)
plt.legend(title="Race Group")
plt.tight_layout()
plt.show()

```



Model 1:

(Supervised) How does general election turnout vary across racial groups and age brackets within Wyoming, and what patterns emerge when these factors intersect?

```

from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler

model_df = df.select(
    col("is_active").alias("label"),
    "race_group",
    "age_group",
    "County"
).dropna()

# Index categorical variables
indexers = [
    StringIndexer(inputCol="race_group", outputCol="race_index"),
    StringIndexer(inputCol="age_group", outputCol="age_index"),
    StringIndexer(inputCol="County", outputCol="county_index")
]

# One-hot encode
encoders = [
    OneHotEncoder(inputCol="race_index", outputCol="race_vec"),
    OneHotEncoder(inputCol="age_index", outputCol="age_vec"),
    OneHotEncoder(inputCol="county_index", outputCol="county_vec")
]

# Combine all features
assembler = VectorAssembler(
    inputCols=["race_vec", "age_vec", "county_vec"],
    outputCol="features"
)

pipeline = Pipeline(stages=indexers + encoders + [assembler])
model_data = pipeline.fit(model_df).transform(model_df)

```

- ▶  model_data: pyspark.sql.dataframe.DataFrame
- ▶  model_df: pyspark.sql.dataframe.DataFrame = [label: integer, race_group: string ... 2 more fields]

```

from pyspark.ml.classification import LogisticRegression

train_data, test_data = model_data.randomSplit([0.7, 0.3], seed=42)

log_reg = LogisticRegression(featuresCol="features", labelCol="label")
log_model = log_reg.fit(train_data)
predictions = log_model.transform(test_data)

```

- ▶  predictions: pyspark.sql.dataframe.DataFrame
- ▶  test_data: pyspark.sql.dataframe.DataFrame
- ▶  train_data: pyspark.sql.dataframe.DataFrame


```

from pyspark.ml.classification import LogisticRegression

train_data, test_data = model_df.randomSplit([0.7, 0.3], seed=42)

train_pipeline_model = pipeline.fit(train_data)
train_transformed = train_pipeline_model.transform(train_data)
test_transformed = train_pipeline_model.transform(test_data)

log_reg = LogisticRegression(featuresCol="features", labelCol="label")

log_model = log_reg.fit(train_transformed)

```

```

▶ test_data: pyspark.sql.dataframe.DataFrame = [label: integer, race_group: string ... 2 more fields]
▶ test_transformed: pyspark.sql.dataframe.DataFrame
▶ train_data: pyspark.sql.dataframe.DataFrame = [label: integer, race_group: string ... 2 more fields]
▶ train_transformed: pyspark.sql.dataframe.DataFrame

```

```

from pyspark.ml.evaluation import MulticlassClassificationEvaluator

accuracy_evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction",
metricName="accuracy")

predictions = log_model.transform(test_transformed)
accuracy = accuracy_evaluator.evaluate(predictions)

print(accuracy)

```

```

▶ predictions: pyspark.sql.dataframe.DataFrame
0.982131380829967

```

Conclusion

Our Supervised Model predicts the voter turnout rate based on the Number of Active Voters in the state of Wyoming from 2000 to 2018. The dataset has 290408, but was trimmed down to 127101, which were analyzed. This model accurately predicts the voter turnout for the upcoming elections in Wyoming useful for data-driven campaigning, as the accuracy of the model was tested at 98.2%. This can help predict the voter total and turnout to create an order of priority; the state of Wyoming can be placed in future elections. The conclusion we got from this model are highly relevant for data-driven campaigning. By identifying demographic and geographic patterns associated with turnout, political strategists can better prioritize outreach efforts and allocate resources more effectively across counties in Wyoming. This allows campaigns to focus on key areas and populations that may influence overall election outcomes.

(Unsupervised) What distinct groups or patterns emerge when clustering Wyoming residents based on homeownership status, racial background, and other demographic features - and how do these clusters differ in their voting behavior?

```
#Converting numerical and categorical values

from pyspark.sql.functions import when, col, regexp_replace

# Categorizing Home Owner Status
df = df.withColumn(
    'Home_Owner_Encoded',
    when(col('CommercialDataLL_Home_Owner_Or_Renter') == 'Likely Renter', 0)
    .when(col('CommercialDataLL_Home_Owner_Or_Renter') == 'Likely Homeowner', 1)
)

# Categorizing Race Group
df = df.withColumn(
    'race_group_encoded',
    when(col('race_group') == 'Other', 0)
    .when(col('race_group') == 'White', 1)
    .when(col('race_group') == 'Hispanic', 2)
    .when(col('race_group') == 'Black', 3)
    .when(col('race_group') == 'Asian', 4)
)

# Normalizing Age
df = df.withColumn('Voters_Age_normalized', (col('Voters_Age') - df.agg({'Voters_Age': 'min'}).first()[0]) /
(df.agg({'Voters_Age': 'max'}).first()[0] - df.agg({'Voters_Age': 'min'}).first()[0]))

# Numerical Conversion
df = df.withColumn('EstimatedIncome_numeric', regexp_replace(col('CommercialData_EstimatedHHIncomeAmount'),
['\$', ','], '').cast('float'))

df = df.withColumn(
    'Voters_Active_encoded',
    when(col('Voters_Active') == 'A', 1)
    .when(col('Voters_Active') == 'I', 0)
)
```

▶  df: pyspark.sql.dataframe.DataFrame = [SEQUENCE: long, LALVOTERID: string ... 455 more fields]

<>:25: SyntaxWarning:

invalid escape sequence '\\$'

<>:25: SyntaxWarning:

invalid escape sequence '\\$'

/root/.ipykernel/14287/command-1289750558566056-1369387408:25: SyntaxWarning:

invalid escape sequence '\\$'

<unknown>:25: SyntaxWarning:

invalid escape sequence '\\$'

```
# K-means Test
from pyspark.ml.feature import VectorAssembler

from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import ClusteringEvaluator

features = ['Home_Owner_Encoded', 'race_group_encoded', 'Voters_Age_normalized', 'EstimatedIncome_numeric']

# Assemble into a single vector column
assembler = VectorAssembler(inputCols=features, outputCol="features", handleInvalid='skip')
df_kmeans = assembler.transform(df)

kmeans = KMeans(featuresCol='features', predictionCol='cluster', k=3, seed=1)
model = kmeans.fit(df_kmeans)
df_clustered = model.transform(df_kmeans)

evaluator = ClusteringEvaluator(featuresCol='features', predictionCol='cluster', metricName='silhouette')
silhouette = evaluator.evaluate(df_clustered)
print(f"Silhouette score: {silhouette}")

df_clustered.groupBy('cluster').agg(
    {'Voters_Active_encoded': 'avg',
     'Home_Owner_Encoded': 'avg',
     'race_group_encoded': 'avg',
     'Voters_Age_normalized': 'avg',
     'EstimatedIncome_numeric': 'avg'}
).show()
```

df_clustered: pyspark.sql.dataframe.DataFrame

df_kmeans: pyspark.sql.dataframe.DataFrame

Silhouette score: 0.7272777875876337

cluster	avg(Home_Owner_Encoded)	avg(Voters_Active_encoded)	avg(Voters_Age_normalized)	avg(race_group_encoded)	avg(EstimatedIncome_numeric)
1	0.8190878675823209	0.9884486705405882	0.5702185043275004	0.970329283324529	50243.73357985561
2	0.925567278508455	0.990099725393843	0.4573473155007033	0.9485474779592427	214314.13499060558
0	0.8899297009190232	0.9880296778991731	0.4820551638096535	0.9586882226904893	110902.93763391355

```
# Bar Plot for 3 Clusters

import pandas as pd

summary_df = df_clustered.groupBy('cluster').agg(
    {'Voters_Active_encoded': 'avg',
     'Home_Owner_Encoded': 'avg',
     'race_group_encoded': 'avg',
     'Voters_Age_normalized': 'avg'}
).orderBy('cluster')

summary_pd = summary_df.toPandas()

# Rename columns for better visual labels
summary_pd.rename(columns={
    'avg(Home_Owner_Encoded)': 'Home Ownership',
    'avg(Voters_Active)': 'Voter Inactivity',
    'avg(Voters_Age_normalized)': 'Age (normalized)',
    'avg(race_group_encoded)': 'Race Group',
    'cluster': 'Cluster'
}, inplace=True)

# Melt to long format
summary_melted = summary_pd.melt(id_vars=['Cluster'], var_name='Feature', value_name='Average')
```

- ▶ summary_df: pyspark.sql.dataframe.DataFrame = [cluster: integer, avg(Voters_Active_encoded): double ... 3 more fields]
- ▶ summary_melted: pandas.core.frame.DataFrame = [Cluster: int32, Feature: object ... 1 more field]
- ▶ summary_pd: pandas.core.frame.DataFrame = [Cluster: int32, avg(Voters_Active_encoded): float64 ... 3 more fields]

```
plt.figure(figsize=(12, 6))
sns.barplot(data=summary_melted, x='Feature', y='Average', hue='Cluster', palette='Set2')

plt.title('Average Feature Values by Cluster')
plt.xticks(rotation=45)
plt.ylabel('Average')
plt.xlabel('Feature')
plt.legend(title='Cluster')
plt.tight_layout()
plt.show()
```



```

# Pie Chart for 3 Clusters

import matplotlib.pyplot as plt
from pyspark.sql.functions import count

# Race Counts
race_counts_df = df_clustered.groupBy('cluster',
'race_group_encoded').agg(count('*').alias('count')).orderBy('cluster')
race_counts_pd = race_counts_df.toPandas()

clusters = race_counts_pd['cluster'].unique()

# Get unique race categories and define colors for each
race_categories = sorted(race_counts_pd['race_group_encoded'].unique())
colors_list = plt.cm.tab10.colors

# Map each race category to a specific color
color_map = {race: colors_list[i % len(colors_list)] for i, race in enumerate(race_categories)}

fig, axes = plt.subplots(1, len(clusters), figsize=(6 * len(clusters), 6))

if len(clusters) == 1:
    axes = [axes]

for ax, cluster in zip(axes, clusters):
    data = race_counts_pd[race_counts_pd['cluster'] == cluster]

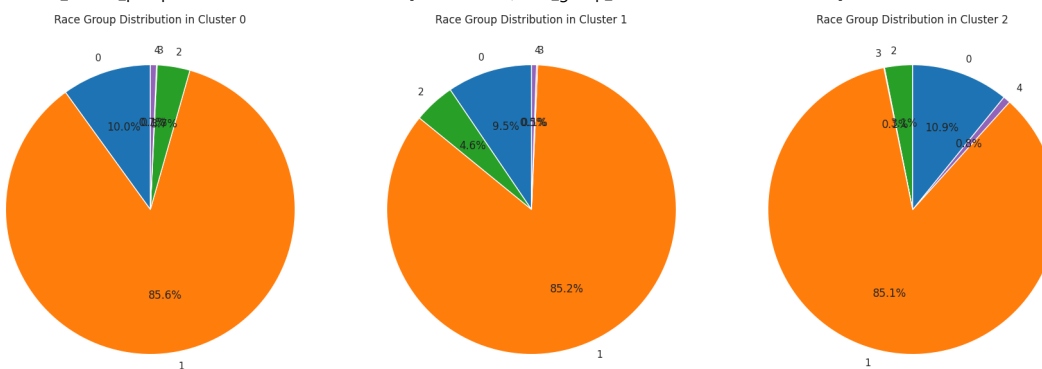
    # Order colors for this cluster's race categories
    pie_colors = [color_map[race] for race in data['race_group_encoded']]

    ax.pie(data['count'], labels=data['race_group_encoded'], autopct='%1.1f%%', startangle=90,
    colors=pie_colors)
    ax.set_title(f'Race Group Distribution in Cluster {cluster}')

plt.tight_layout()
plt.show()

```

- data: pandas.core.frame.DataFrame = [cluster: int32, race_group_encoded: int32 ... 1 more field]
- race_counts_df: pyspark.sql.dataframe.DataFrame = [cluster: integer, race_group_encoded: integer ... 1 more field]
- race_counts_pd: pandas.core.frame.DataFrame = [cluster: int32, race_group_encoded: int32 ... 1 more field]



```

# Prep for voting turnout rate throughout the years

df = df_clustered

# Turnout rate calculation grouped by 'cluster'
from pyspark.sql.functions import col, when, avg, lit
from functools import reduce

years = [2004, 2006, 2008, 2010, 2012, 2014, 2016, 2018]
year_cols = [f"General_{year}" for year in years]

turnout_rates = []

for year_col in year_cols:
    voted_col = f"voted_{year_col[-4:]}"
    df = df.withColumn(voted_col, when(col(year_col) == "Y", 1).otherwise(0))

    turnout_by_cluster = (
        df.groupBy("cluster")
        .agg(avg(voted_col).alias("turnout_rate"))
        .withColumn("year", lit(int(year_col[-4:])))
    )

    turnout_rates.append(turnout_by_cluster)

turnout_df = reduce(lambda df1, df2: df1.union(df2), turnout_rates)

```

- ▶  df: pyspark.sql.dataframe.DataFrame
- ▶  turnout_by_cluster: pyspark.sql.dataframe.DataFrame = [cluster: integer, turnout_rate: double ... 1 more field]
- ▶  turnout_df: pyspark.sql.dataframe.DataFrame = [cluster: integer, turnout_rate: double ... 1 more field]

```

# Voting turnout rate throughout the years

import matplotlib.pyplot as plt
import seaborn as sns

# Collect to pandas DataFrame
pdf = turnout_df.toPandas()

pdf['year'] = pdf['year'].astype(int)

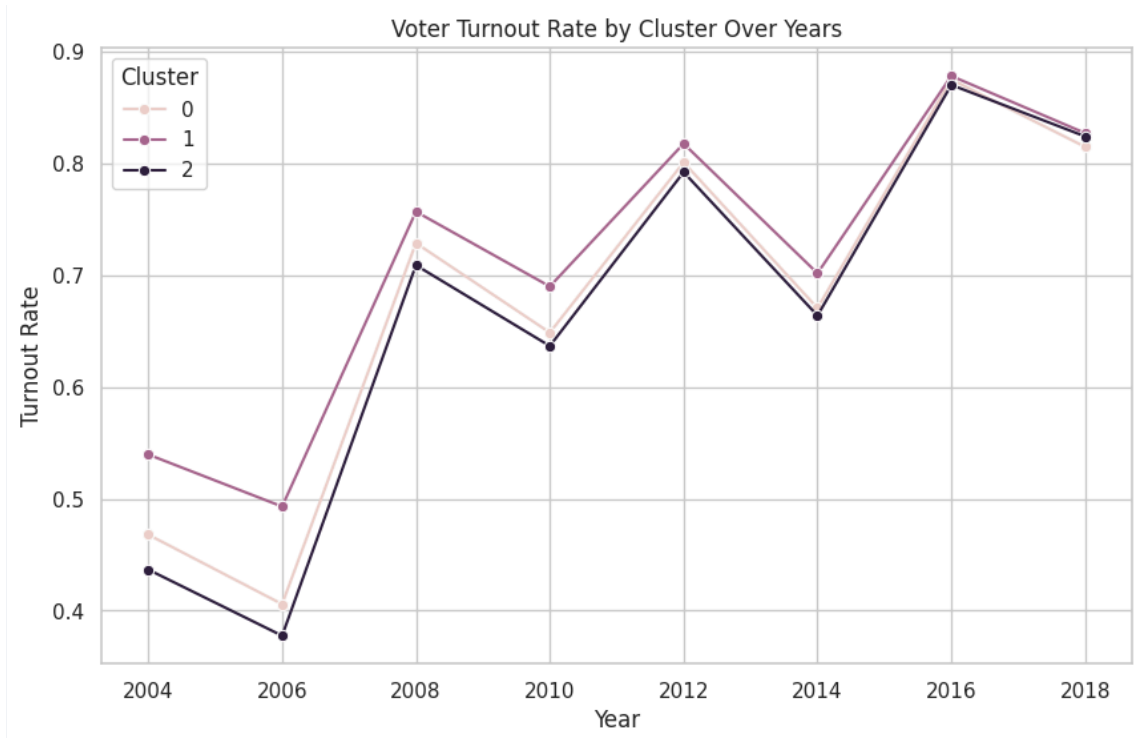
plt.figure(figsize=(10, 6))

# Lineplot
sns.lineplot(data=pdf, x='year', y='turnout_rate', hue='cluster', marker='o')

plt.title('Voter Turnout Rate by Cluster Over Years')
plt.xlabel('Year')
plt.ylabel('Turnout Rate')
plt.legend(title='Cluster')
plt.grid(True)
plt.show()

```

- ▶  pdf: pandas.core.frame.DataFrame = [cluster: int32, turnout_rate: float64 ... 1 more field]



Interpretation & Summary: We performed K-means clustering to separate 3 distinct groups on the dataset based on Wyoming residents to differentiate income level, age, home ownership status, and political activity.

Cluster 0: The oldest and lowest-income group. Lowest HomeOwner Percentage, and mostly race group 1. 98.84% voting rate

Cluster 1: Sits in the middle for both income and age. Middle HomeOwner Percentage, and mostly race group 1. 98.80% voting rate

Cluster 2: The wealthiest group with the youngest voters. Highest HomeOwner Percentage, and mostly race group 1. 99.00% voting rate

Answering the Question: The clusters slightly differ by age and income. Race composition remains largely consistent across clusters, and racial background does not seem to be a major factor due to the demographic homogeneity (Majority White). Overall, voting behavior differences among clusters are subtle but indicate that higher-income, younger homeowners tend to be a bit more politically active.